



BCS202L- Data Structures and Algorithms

Dr. Priyanka N

Assistant Professor Senior Grade I

**School of Computer Science & Engineering
VIT,Vellore.**



BCS202L- Data Structures and Algorithms

- **Module-1: Algorithm Analysis**
- **Module-2: Linear Data Structures**
- **Module-3: Searching and Sorting**
- **Module-4: Trees**
- **Module-5: Graphs**
- **Module-6: Hashing**
- **Module-7: Heaps and AVL Trees**

Module:1	Algorithm Analysis	8 hours
Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth - Algorithm efficiency – best case, worst case, average case - Analysis of non-recursive and recursive algorithms - Asymptotic analysis for recurrence relation: Iteration Method, Substitution Method, Master Method and Recursive Tree Method.		
Module:2	Linear Data Structures	7 hours
Arrays: 1D and 2D array- Stack - Applications of stack: Expression Evaluation, Conversion of Infix to postfix and prefix expression, Tower of Hanoi – Queue - Types of Queue: Circular Queue, Double Ended Queue (deQueue) - Applications – List: Singly linked lists, Doubly linked lists, Circular linked lists- Applications: Polynomial Manipulation.		
Module:3	Searching and Sorting	7 hours
Searching: Linear Search and binary search – Applications. Sorting: Insertion sort, Selection sort, Bubble sort, Counting sort, Quick sort, Merge sort - Analysis of sorting algorithms.		
Module:4	Trees	6 hours
Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the k^{th} minimum element.		
Module:5	Graphs	6 hours
Terminology – Representation of Graph – Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.		
Module:6	Hashing	4 hours
Hash functions - Separate chaining - Open hashing: Linear probing, Quadratic probing, Double hashing - Closed hashing - Random probing – Rehashing - Extendible hashing.		
Module:7	Heaps and AVL Trees	5 hours
Heaps - Heap sort- Applications -Priority Queue using Heaps. AVL trees: Terminology, basic operations (rotation, insertion and deletion).		

BCS202L- Data Structures and Algorithms

Text Books:

1. Mark A. Weiss, Data Structures & Algorithm Analysis in C++, 4 th Edition, 2013, Pearson Education.

Reference Books:

1. Alfred V. Aho, Jeffrey D. Ullman and John E. Hopcroft, Data Structures and Algorithms, 1983, Pearson Education.
2. Horowitz, Sahni and S. Anderson-Freed, Fundamentals of Data Structures in C, 2008, 2nd Edition, Universities Press.
3. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

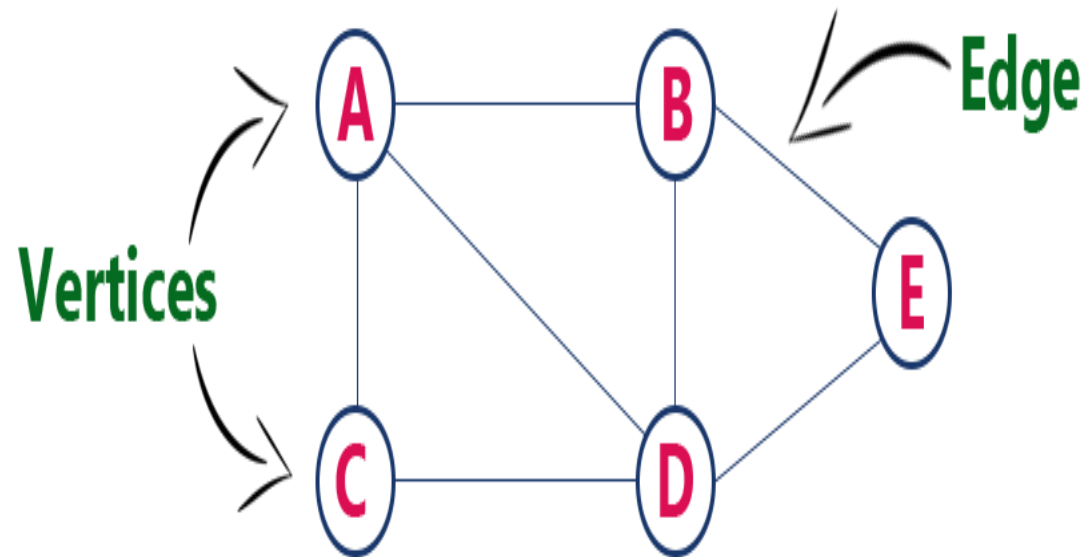
Module 5: Graphs(6 Hours)

- Terminology and Representation
- Graph Traversal
 - DFS
 - BFS
- Minimum Spanning Tree
 - Prim's Algorithm
 - Kruskal's Algorithm
- Single Source Shortest Path
 - Dijkstra's Algorithm

Graph- Definition

- Graph is a non-linear data structure.
- It contains a set of points known as **nodes (or vertices)** and a set of links known as **edges (or Arcs)**. Here edges are used to connect the vertices.
- A graph is defined as “**G**” consist of two sets **V** and **E** where **V** is a finite set of non empty set of vertices and **E** is set of pairs of vertices. These pairs are called edges.
- Generally, a graph **G** is represented as **$G = (V, E)$** , where **V is set of vertices** and **E is set of edges**.

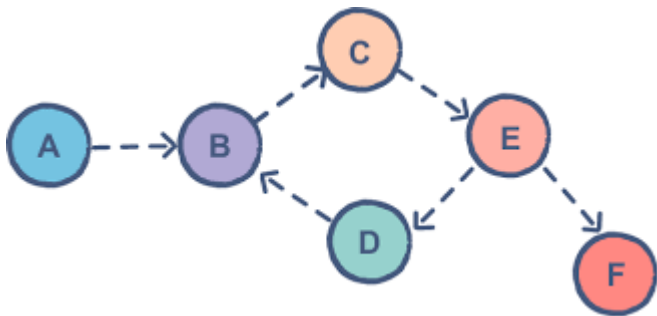
Graph- Example – 5 vertices and 7 Edges



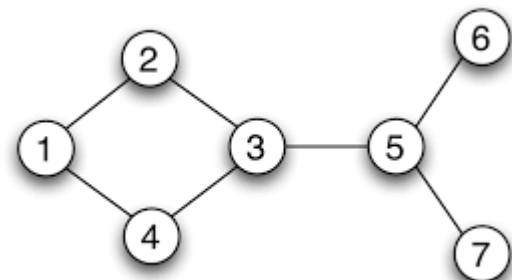
Graph Terminology

- **Types of Graph**

- Directed: A graph with only directed edges is said to be directed graph.
- Undirected Graph: A graph with only undirected edges is said to be undirected graph.



Directed Graph

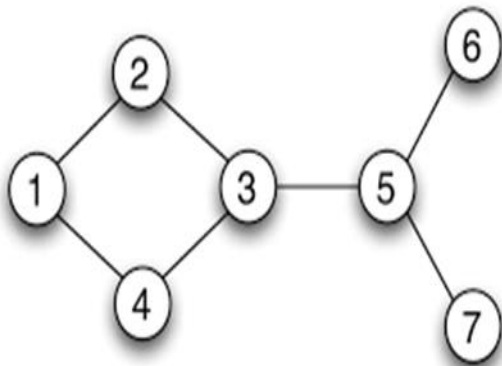


Un Directed Graph

Graph Terminology

- **Vertex**

- Individual data element of a graph is called as Vertex. **Vertex** is also known as **node**. In the below example graph,



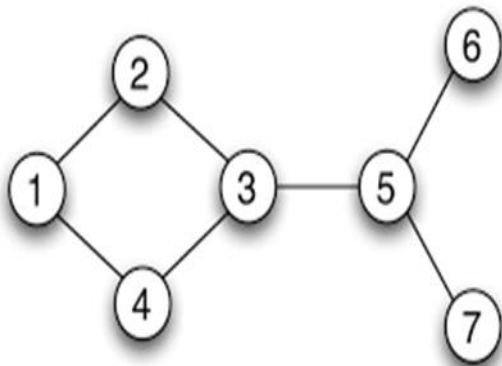
For Example:

1,2,3,4,5,6,7 are known as vertices.

Graph Terminology

- **Edges**

- An edge is a connecting link between two vertices. **Edge** is also known as **Arc**. An edge is represented as (startingVertex, endingVertex).



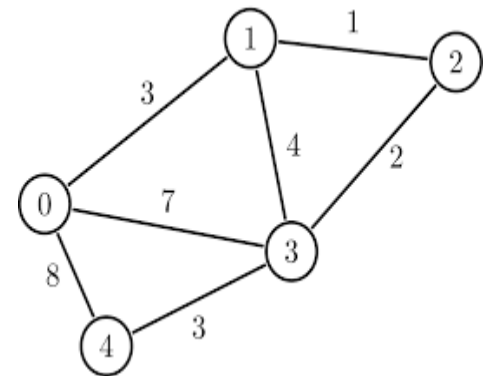
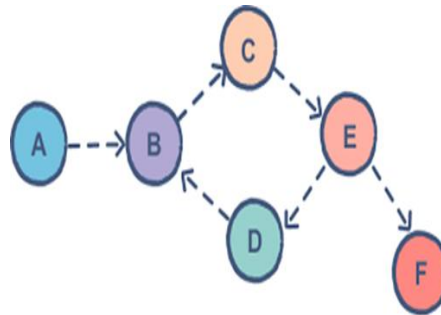
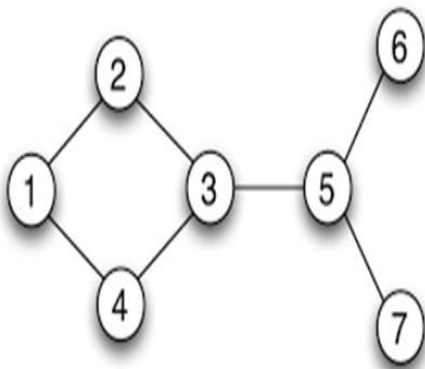
For Example:

The edges in the graphs are (1,2),(1,4),(2,3),(3,4),(3,5),(5,6), (5,7)

Graph Terminology

- **Types of Edges**

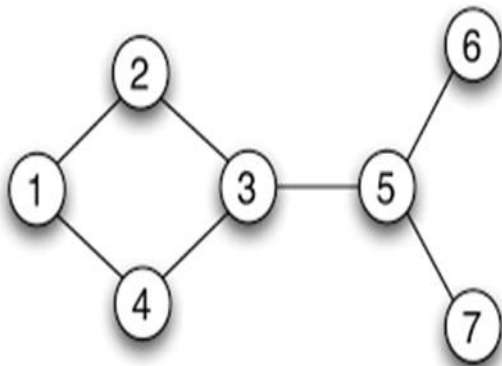
- **Undirected Edge** - An undirected edge is a bidirectional edge. If there is undirected edge between vertices 1 and 2 then edge (1 , 2) is equal to edge (2 , 1).
- **Directed Edge** - A directed edge is a unidirectional edge. If there is directed edge between vertices 1 and 2 then edge (1 , 2) is not equal to edge (2 , 1).
- **Weighted Edge** - A weighted edge is a edge with value (cost) on it.



Graph Terminology

- **Adjacent**

- If there is an edge between vertices A and B then both A and B are said to be adjacent. In other words, vertices A and B are said to be adjacent if there is an edge between them.

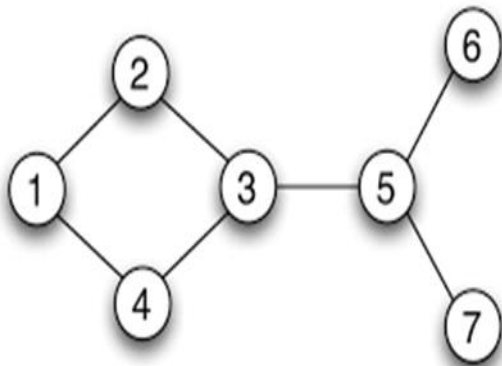


For Example:
Adjacent Nodes
for 3 is (2,4,5)

Graph Terminology

- **Incident**

- Edge is said to be incident on a vertex if the vertex is one of the endpoints of that edge.

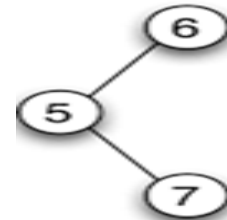
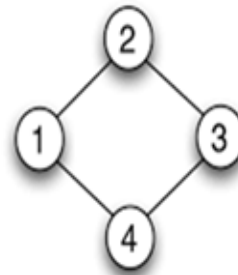
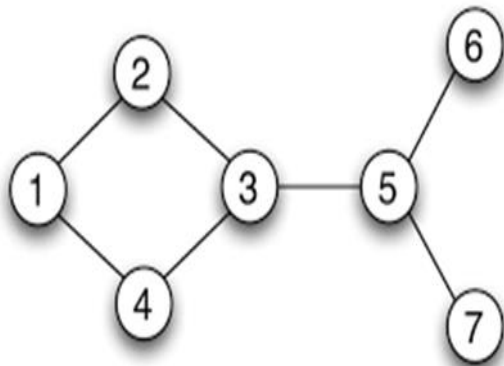


For Example: The edges incident on Vertex 2 is (1,2) and (2,3)

Graph Terminology

- **Subgraph**

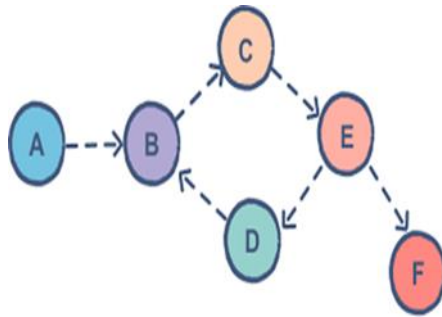
- A Sub graph of G is G' such that vertices of G' belongs to G and all edges of G' belongs to G .



Graph Terminology

- **Path**

- A path is a sequence of alternate vertices and edges that starts at a vertex and ends at other vertex such that each edge is incident to its predecessor and successor vertex.
- Length of the path: Number of edges on the path.



Example Path:

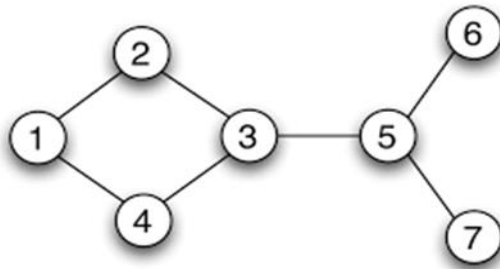
A-B-C-E-F

- Cycle: It is a simple path in which first and last vertices are same.

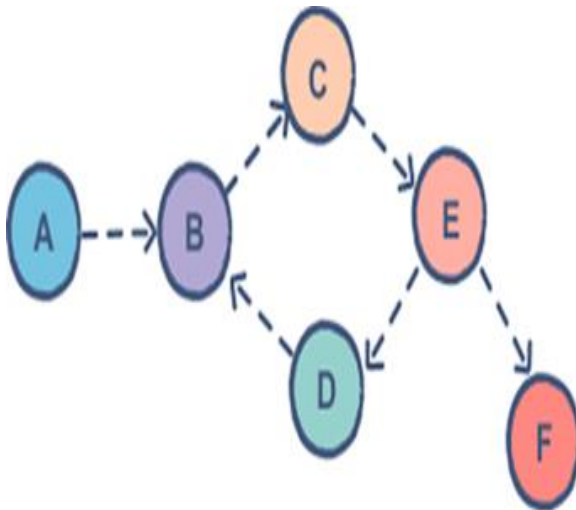
Graph Terminology

- **Degree of the Vertex**

- Total number of edges connected to a vertex is said to be degree of that vertex.



Example: Vertex of 5 is 3



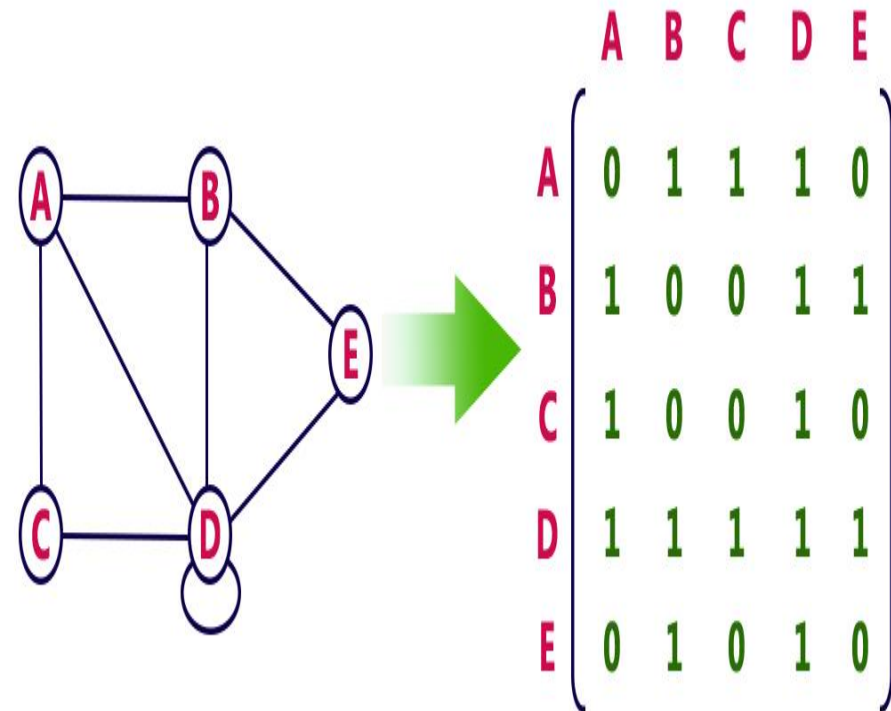
InDegree : Total number of incoming edges connected to a vertex is said to be indegree of that vertex.

OutDegree : Total number of Outgoing edges connected to a vertex is said to be indegree of that vertex.

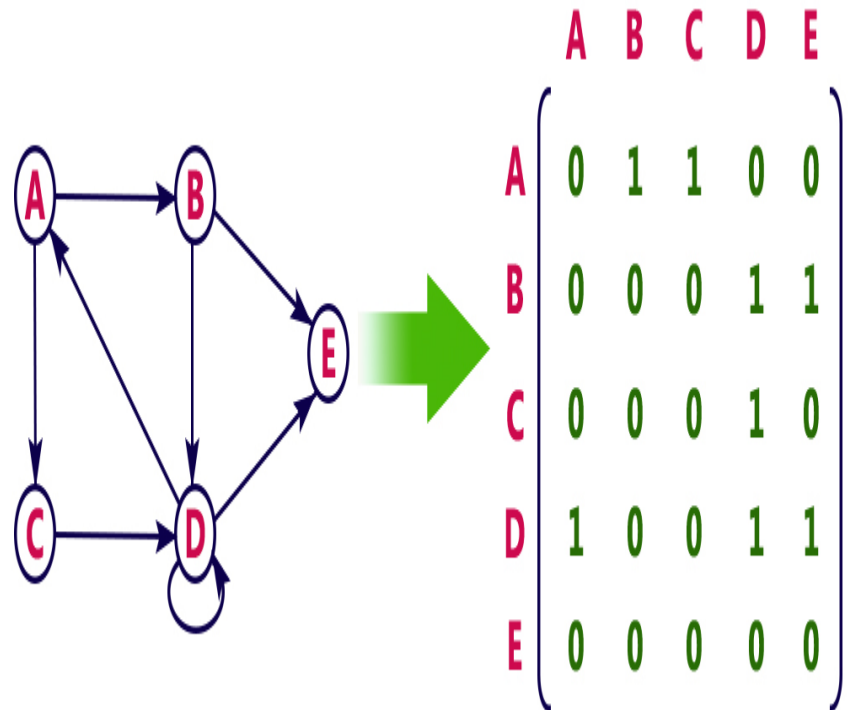
Representation of Graphs

- **Adjacency Matrix**

Undirected Graph



Directed Graph



Representation of Graphs

- Adjacency List

